

Assignment 3

Name: Nisarg Upadhyay
Date: 22 February 2026

1. Links

Resource	Link
HuggingFace Model	NisargUpadhyay/distilbert-goodreads-genres
GitHub Repository	NisargUpadhyayIITJ/MLOps-NisargUpadhyay-B23CS1075
Original Notebook	Colab Link
Dataset	UCSD Book Graph

2. Model Selection

Chosen Model: `distilbert-base-cased`

DistilBERT is a distilled (compressed) version of Google's BERT, developed by HuggingFace. It was selected for this assignment for the following reasons:

Criterion	Justification
Efficiency	40% fewer parameters than BERT-base (66M vs 110M), making it faster to train and deploy
Speed	~60% faster inference than BERT-base — ideal for iterative experiments and Docker containers
Performance	Retains 97% of BERT's language understanding capability (as measured on GLUE benchmark)
Cased Variant	Preserves capitalization, which can carry semantic meaning in book reviews (e.g., proper nouns, genre-specific terms)
Task Fit	<code>DistilBertForSequenceClassification</code> provides a ready-made classification head, perfect for our multi-class genre classification task
Ecosystem	First-class support in HuggingFace: Trainer API, tokenizer, one-command push to Hub
Resource Constraints	Small enough to fine-tune on a single GPU in minutes and to package into a Docker image for evaluation

The model was loaded with 8 output labels corresponding to the Goodreads book genres.

3. Training Summary

Dataset

- **Source:** [UCSD Book Graph](#) — Goodreads book reviews
- **Data Loading:** Reviews are streamed from gzipped JSON files hosted at UCSD; 10,000 reviews per genre are read, 2,000 randomly sampled, then 1,000 used per genre (800 train / 200 test)
- **Genres (8 classes):** Children, Comics & Graphic, Fantasy & Paranormal, History & Biography, Mystery Thriller & Crime, Poetry, Romance, Young Adult

Split	Total Samples	Per Genre
Train	6,400	800
Test	1,600	200

Hyperparameters

Parameter	Value
Epochs	3
Learning Rate	5e-5 (AdamW optimizer)
Train Batch Size	10 per device
Eval Batch Size	16 per device
Warmup Steps	100
Weight Decay	0.01
Max Sequence Length	512 tokens
Evaluation Strategy	Every 100 steps

Training Results (evaluated on the same test split used during training)

Metric	Value
Loss	1.2379
Accuracy	59.88%
Precision	60.44%
Recall	59.88%
F1 (weighted)	60.01%

4. Evaluation Comparison — Local vs. HuggingFace Hub

After training, we evaluated the model from two sources:

1. **Local model** — loaded from `./distilbert-reviews-genres/` (the saved checkpoint)
2. **Hub model** — loaded from `NisargUpadhyay/distilbert-goodreads-genres`

Both evaluations used **freshly sampled** test data (different random sample from the same URLs), which explains why the metrics differ from the training evaluation.

Side-by-Side Comparison

Metric	Training Eval	Local Model Eval	Hub Model Eval
Loss	1.2379	-6.4579	-6.6409
Accuracy	59.88%	36.75%	36.56%
Precision	60.44%	42.50%	45.28%
Recall	59.88%	36.75%	36.56%
F1 (weighted)	60.01%	37.71%	38.38%

Per-Genre Classification Report (Local Model)

Genre	Precision	Recall	F1-Score	Support
children	0.38	0.51	0.44	200
comics_graphic	0.58	0.39	0.46	200
fantasy_paranormal	0.28	0.20	0.23	200
history_biography	0.48	0.33	0.39	200
mystery_thriller_crime	0.47	0.30	0.37	200
poetry	0.66	0.47	0.55	200
romance	0.19	0.48	0.27	200
young_adult	0.35	0.26	0.30	200
accuracy			0.37	1600
macro avg	0.43	0.37	0.38	1600
weighted avg	0.43	0.37	0.38	1600

Per-Genre Classification Report (Hub Model)

Genre	Precision	Recall	F1-Score	Support
children	0.41	0.51	0.45	200
comics_graphic	0.59	0.35	0.44	200
fantasy_paranormal	0.34	0.20	0.26	200
history_biography	0.46	0.30	0.36	200
mystery_thriller_crime	0.52	0.33	0.40	200
poetry	0.74	0.48	0.58	200

Genre	Precision	Recall	F1-Score	Support
romance	0.16	0.47	0.24	200
young_adult	0.39	0.28	0.33	200
accuracy			0.37	1600
macro avg	0.45	0.37	0.38	1600
weighted avg	0.45	0.37	0.38	1600

Analysis

- **Local $\approx$ Hub:** The local and Hub model metrics are nearly identical (within ~1%), confirming that the model was correctly uploaded and downloaded from HuggingFace Hub. The small differences are due to different random test data samples.
- **Training eval vs. post-training eval:** The training evaluation (59.88% accuracy) was performed on the *same* test split during training. The post-training evaluations (36-37% accuracy) used a *freshly downloaded and re-sampled* set of reviews, which introduces variance — different reviews, different difficulty.
- **Best/worst genres (consistent across runs):** Poetry and Comics & Graphic are consistently the easiest to classify (distinctive language), while Fantasy & Paranormal and Young Adult remain hardest (overlapping vocabulary with other genres).

5. Docker Image Build Instructions

Prerequisites

- Docker installed ([Get Docker](#))
- NVIDIA Container Toolkit (for GPU training — optional for CPU-only eval)

Image 1: Training Image ([Dockerfile](#))

This image sets up the full training environment.

```
# Build the image
docker build -t mlops-train .

# Run: Train model (CPU)
docker run mlops-train

# Run: Train model (GPU)
docker run --gpus all mlops-train

# Run: Train + push to HuggingFace Hub
docker run --gpus all \
  -e HF_TOKEN=<your_huggingface_token> \
  mlops-train \
  python src/train.py --push_to_hub --hf_username NisargUpadhyay
```

```
# View real-time logs
docker run mlops-train      # Foreground – logs print directly

# Or run in background and check logs
docker run -d --name train-container mlops-train
docker logs -f train-container
```

Dockerfile overview:

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ src/
CMD ["python", "src/train.py"]
```

Image 2: Production Evaluation Image (`Dockerfile.eval`)

This lightweight image automatically pulls the model from HuggingFace Hub and runs evaluation on startup.

```
# Build the image
docker build -f Dockerfile.eval -t mlops-eval .

# Run: Auto-evaluate (pulls model from HF Hub)
docker run mlops-eval

# Run with a custom model repo
docker run mlops-eval python src/eval.py --hf_repo
YOUR_USERNAME/YOUR_MODEL

# Check logs of a background container
docker run -d --name eval-container mlops-eval
docker logs -f eval-container
docker logs --tail 50 eval-container      # Last 50 lines
```

Dockerfile.eval overview:

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/eval.py src/data.py src/utils.py src/
CMD ["python", "src/eval.py", "--hf_repo", "NisargUpadhyay/distilbert-
goodreads-genres"]
```

Command	Purpose
<code>docker build -t <name> .</code>	Build image from Dockerfile
<code>docker build -f <file> -t <name> .</code>	Build from a specific Dockerfile
<code>docker run <image></code>	Run container (foreground)
<code>docker run -d --name <name> <image></code>	Run container (background)
<code>docker logs <container></code>	View container logs
<code>docker logs -f <container></code>	Follow logs in real-time
<code>docker ps -a</code>	List all containers
<code>docker images</code>	List images

6. Challenges Encountered

Challenge 1: Large data downloads

Each genre's gzipped review file is ~1GB; downloading all 8 takes significant time.

Solution: Streamed data with `requests.get(stream=True)` and limited to 10K reviews per genre, sampling 2K.

Challenge 2: Notebook → production conversion

Colab magic commands (`!pip`, `%matplotlib`) don't work in `.py` files.

Solution: Removed all magic commands; restructured into 4 clean modules with proper imports.

Challenge 3: Random test data variance

Re-running evaluation yields different metrics because test data is randomly re-sampled each time.

Solution: Documented this as expected behavior; consistent accuracy observed across same-split evaluations.

Challenge 4: HuggingFace model download throttling

Model download from Hub was very slow/stalled at times due to network throttling.

Solution: Used HuggingFace's built-in caching, retry logic, and verified upload independently.

Challenge 5: WandB API key requirement

Default HuggingFace Trainer tries to log to Weights & Biases, which requires an API key.

Solution: Disabled with `WANDB_DISABLED=true` env variable and `report_to=[]` in TrainingArguments.

Challenge 6: Genre overlap

Fantasy/Paranormal and Young Adult frequently confused by the model.

Solution: This is a real-world limitation of genre boundaries — reviews in these categories share similar language.

Challenge 7: Docker without GPU

The eval image needs to work without GPU access.

Solution: The model is small enough (263MB) to run inference on CPU; `Dockerfile.eval` doesn't require GPU.